Rapport de Projet : Compilateur TPC

Binôme :

Rafael BARBOSA MARTINS

Ugo DEPAOLI

Groupe 8

Répartition du travail : 50-50

1. Introduction

L'objectif de ce projet est de réaliser un compilateur pour le langage TPC (un sous-ensemble du C) en se basant sur le projet d'analyse syntaxique du semestre precèdent. Le programme doit être capable de vérifier la correction lexicale, syntaxique ainsi que sémantique (ou juste des avertissement) d'un fichier source et, en l'absence d'erreurs, de construire et d'afficher l'arbre syntaxique et la table des symboles et de créer le code assembleur correspondant. Une partie centrale du travail consiste à corriger les éventuelles erreurs de l'ancien projet, gérer la traduction de l'arbre vers du code assembleur et verifier les erreurs sémantique (ou non) du code tpc donné ainsi que la création (et l'affichage) de la table des symboles.

2. Architecture du Projet

src/ : Contient nos fichiers sources (le lexer, le parser, les fichier pour faire l'arbre, les fichiers s'occupant de créer le code en assembleur ainsi que les fichiers qui s'occupent de la validité sémantique du code tpc).

bin/ : Contient l'exécutable "tpcc".

obj/ : Contient les fichiers objets et temporaires.

asm/ : Contient les fichiers assembleur correpondant au code tpc testé via l'exécutable

test/ : Contient nos jeux de tests (good/, syn-err/, sem-err/ et warn/).

rep/ : Contient ce rapport.

Le projet est compilé via un Makefile qui génère l'exécutable bin/tpcc.

3. Fonctionnalités :

L'analyseur permet de prendre un fichier en paramètre et nous afficher l'abre correspondant si ce fichier ne contient pas d'erreur.

Ensuite il y a une traduction de cet arbre en code assembleur.

Un fichier peut être mis en input via < ou en étant mis en argument après les options.

Exemple :

- ./bin/tpcc [OPTIONS] FILE.tpc
- ./bin/tpcc [OPTIONS] < FILE.tpc

LES OPTIONS

-h --help permet d'afficher l'aide de la ligne de commande de l'analyseur syntaxique.

-t --tree permet d'afficher l'arbre syntaxique généré.

-s, --symtabs Affiche la table des symboles sur la sortie standard

(si un h est rentré, l'aide sera affiché et tout le reste ignoré)

3.b Fonctionnalités du Makefile

make : pour créer l'exécutable

make clean : pour retirer les fichiers asm/, bin/ et obj/

make test : pour créer l'exécutable et lancer tout les tests pour voir les résultats attendus

make asm : pour créer l'exécutable correspondant au fichier _anonymous.asm

make asm-all : pour créer le fichier exécutable correspondant au ficheirs .asm présent dans asm/

4. Difficultés :

- L'une des plus grande difficultés de ce projet était de gérer les structures, soit dans le parser, ou dans la vérification sémantique
- Une autre difficulté était la génération du code assembleur, qui s'avérait plus compliquée que prevu

5. Utilisaion de l'Inteligence Artificielle

L'IA à été utilisé principalement pour la création des banc de test (qui ont été verifié après) et quelque débugage lorsqu'on ne comprennait pas les erreurs qui nous étaient retournée

Nous avons aussi utilisé l'IA pour des fonctionnalités du makefile (pour la commande [make asm-all])